Lab 6 Report

Basic Information

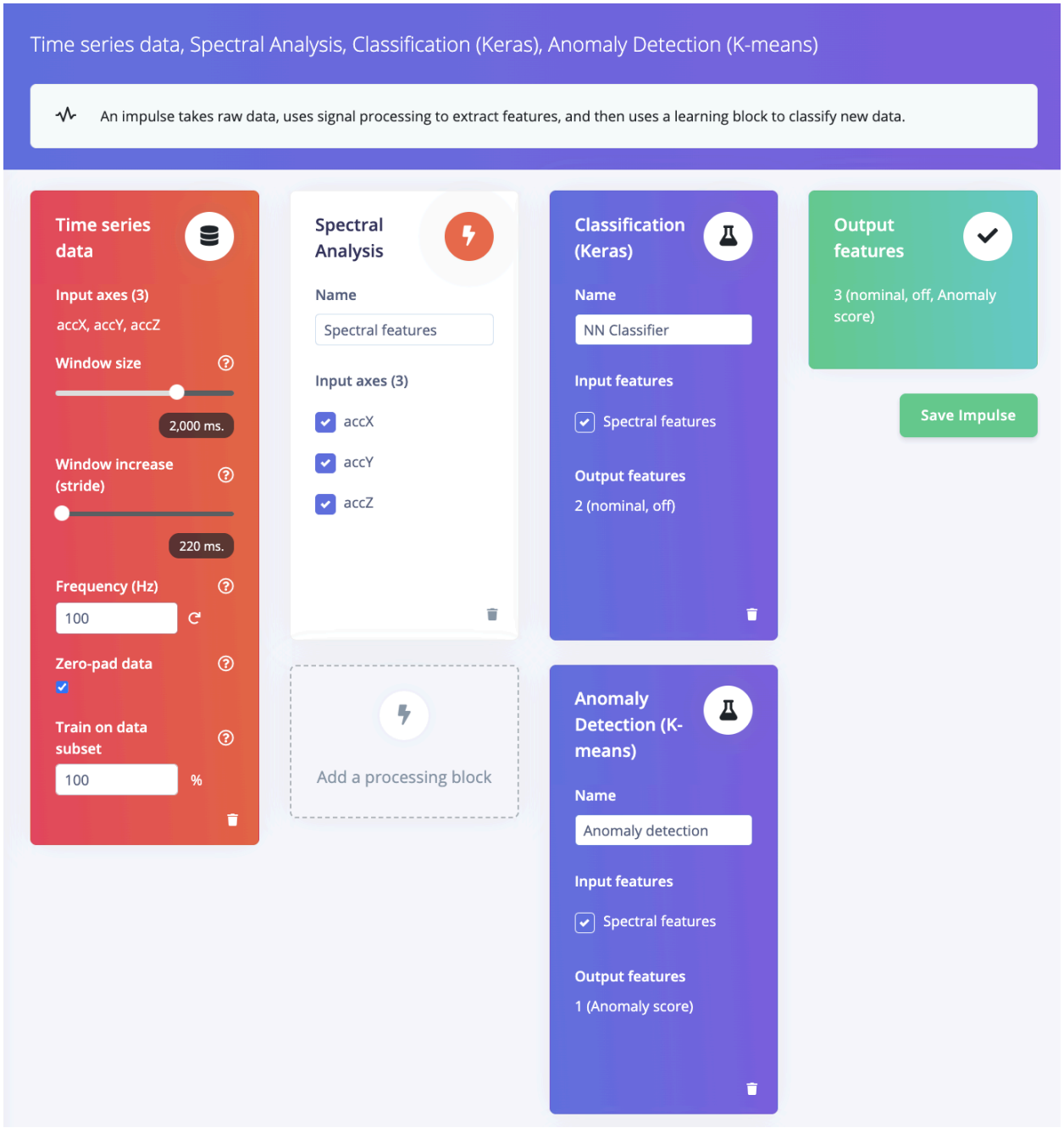
- Name: Richard (Yunchao) He
- Report document: https://docs.google.com/document/d/1apacgu7fLgZoijypdoglyRBu8e16tYf-ixqlIZ47vw4/edit?tab=t.0

Section 1

- Link to the Edge Impulse: https://studio.edgeimpulse.com/studio/1074061

- Answer to the Part 1 Questions
    - 1. Raw inputs: State the raw inputs used in the Edge Impulse project.

The Edge Impulse project uses three-axis accelerometer time-series data as its raw input. The three input axes are **accX, accY, and accZ**, sampled at **100 Hz**. The impulse processes the data using a **2000 ms window** with a **220 ms window increase (stride)**. The raw acceleration data is passed through a Spectral Analysis processing block before being used by the learning blocks.
See the screenshot below:



- 2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

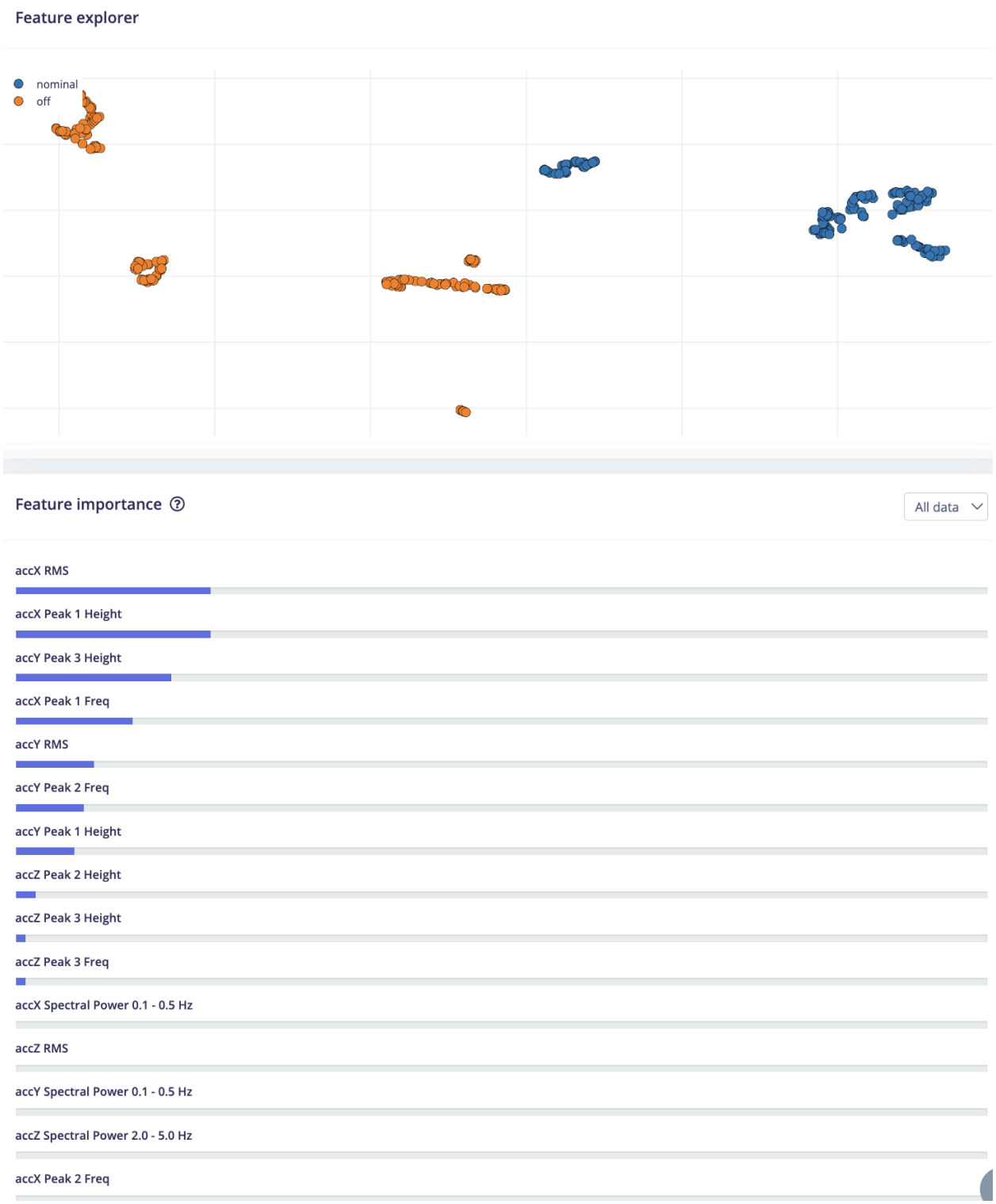
The NN classifier receives **spectral features** generated from the three-axis accelerometer data by the Spectral Analysis DSP block. The classifier has **33 input features**, consisting of 11 features from each of the accX, accY, and accZ axes. These features include RMS, dominant spectral peak frequencies, peak heights, and spectral power over several frequency bands. Example feature names include:

1. accX RMS
2. accX Peak 1 Freq
3. accX Peak 1 Height
4. accX Peak 2 Freq
5. accX Peak 2 Height
6. accX Spectral Power 0.1 - 0.5
7. accY RMS
8. accZ Peak 1 Freq

etc.

The neural network architecture contains a 33-feature input layer, two dense hidden layers with 32 and 16 neurons, and a two-class softmax output layer.

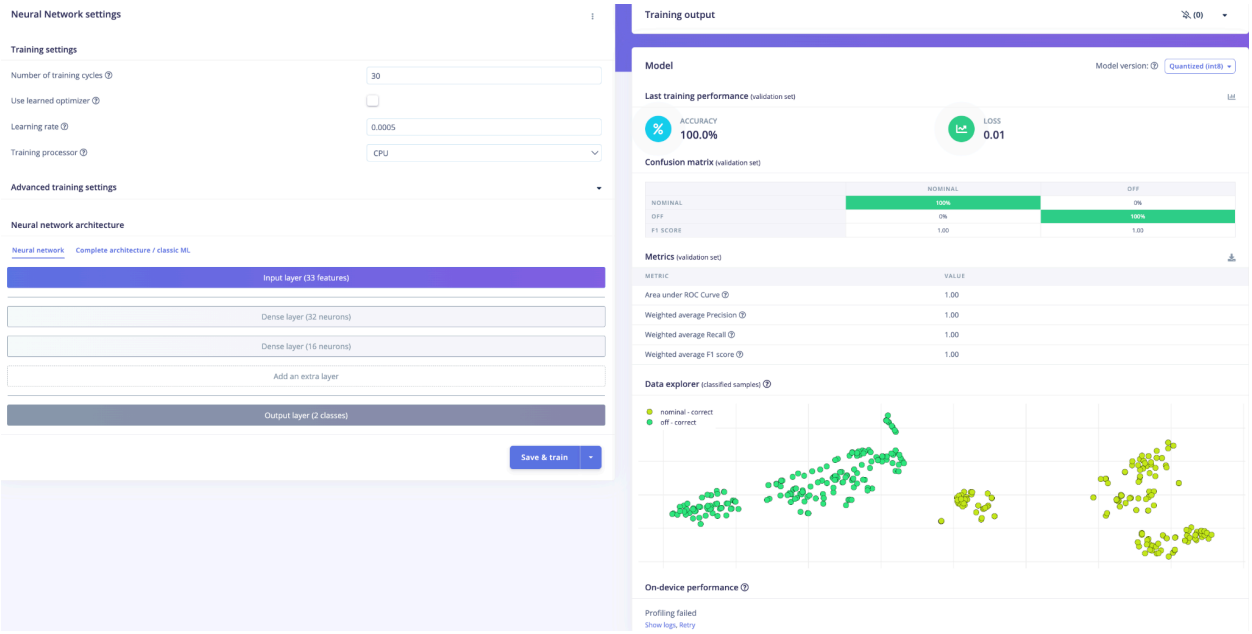
The Figure below shows the most important features:



- 3. NN classifier outputs: State the outputs of the NN classifier.

The NN classifier produces two softmax outputs corresponding to the classes **nominal** and **off**. Each output is a confidence score or probability for the corresponding fan

operating state. The class with the higher score is normally selected as the classifier's prediction.
After training, my model achieved a validation accuracy of **100.0%** and a validation loss of **0.01**. It's so good.
See the screenshot below about training performance, confusion matrix, etc.:



- 4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

The anomaly detector uses **five spectral features** generated from the three-axis accelerometer data by the Spectral Analysis DSP block. The five selected features are:

1. **accX RMS**
2. **accX Peak 1 Freq**
3. **accX Peak 1 Height**
4. **accY Peak 3 Height**
5. **accZ Peak 2 Height**

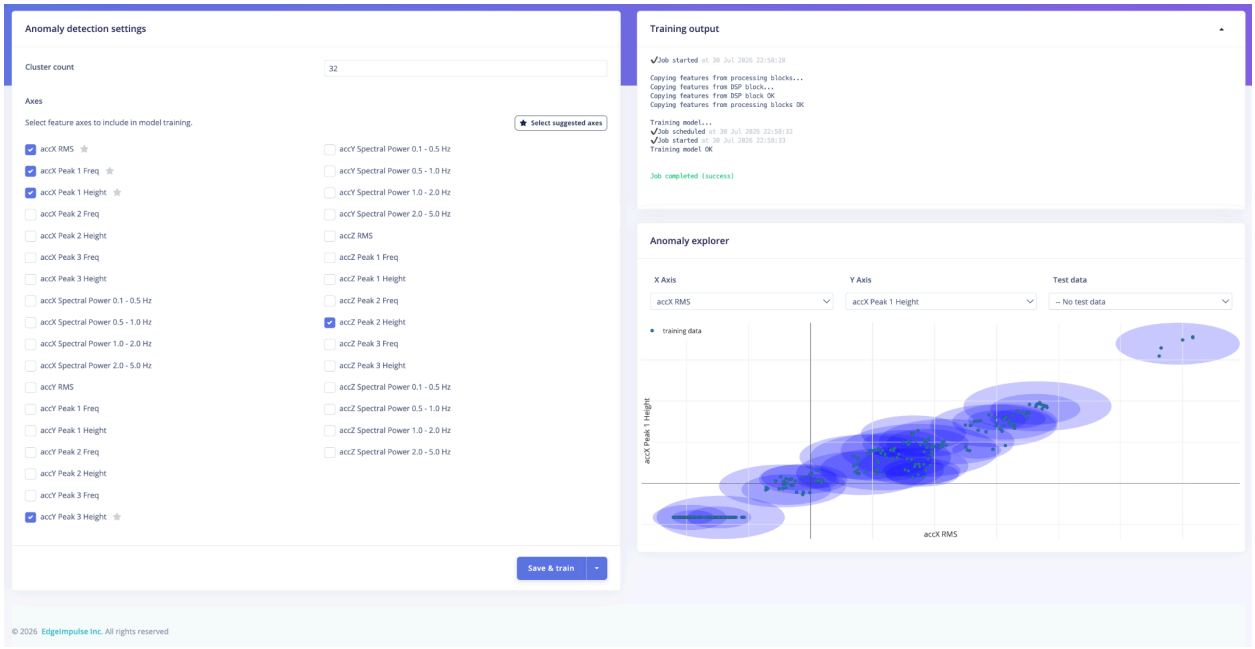
The features were selected based primarily on the feature-importance ranking shown by Edge Impulse. The first four features were among the more important features for separating the fan operating conditions. I also included `accZ Peak 2 Height` so that the anomaly detector contained information from all three accelerometer axes.

These five features were used to train a **K-means anomaly detector with 32 clusters**. The model represents the patterns found in the training data using these clusters. During inference, a new sample that is close to the learned clusters receives a relatively low anomaly score, while

a sample that is far from the clusters receives a higher anomaly score and is more likely to be considered anomalous.

In the Anomaly Explorer, the training samples are covered by multiple learned cluster regions. The displayed graph shows `accX RMS` and `accX Peak 1 Height` as a two-dimensional projection, although the actual anomaly detector was trained using all five selected features.

See the screenshot below:



- 5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

  See the screenshot below:

```
Inferencing settings:
        Interval: 10.00 ms.
        Frame size: 600
        Sample length: 2000 ms.
        No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 609 us, anomaly 601 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.078125
  off: 0.921875
Anomaly prediction: -0.293790
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 611 us, anomaly 631 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 0.864340
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 608 us, anomaly 585 us, postprocessing 50 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 0.858499
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 610 us, anomaly 584 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 0.861623
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 611 us, anomaly 585 us, postprocessing 77 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 0.860010
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 610 us, anomaly 615 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 0.862336
Starting inferencing in 2 seconds...
```

- 6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

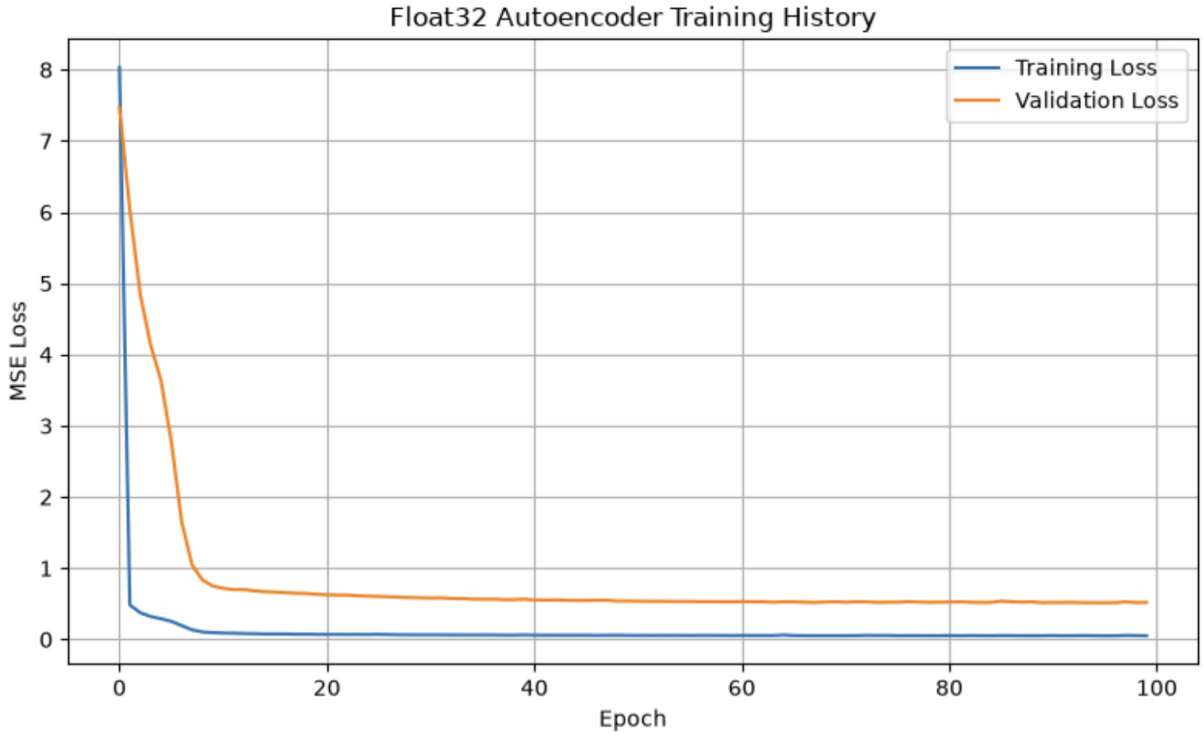
The model was successfully deployed and ran correctly on the Arduino Nano 33 BLE Sense. When the board was placed stationary on a table, the classifier generally favored the `off` class, which is reasonable because little vibration was detected. However, the later outputs (`off` ≈ `0.56`, `nominal` ≈ `0.44`) were close, indicating low classification confidence. The anomaly score of approximately 0.86 also suggests that the tabletop sensor data may differ from the training distribution.

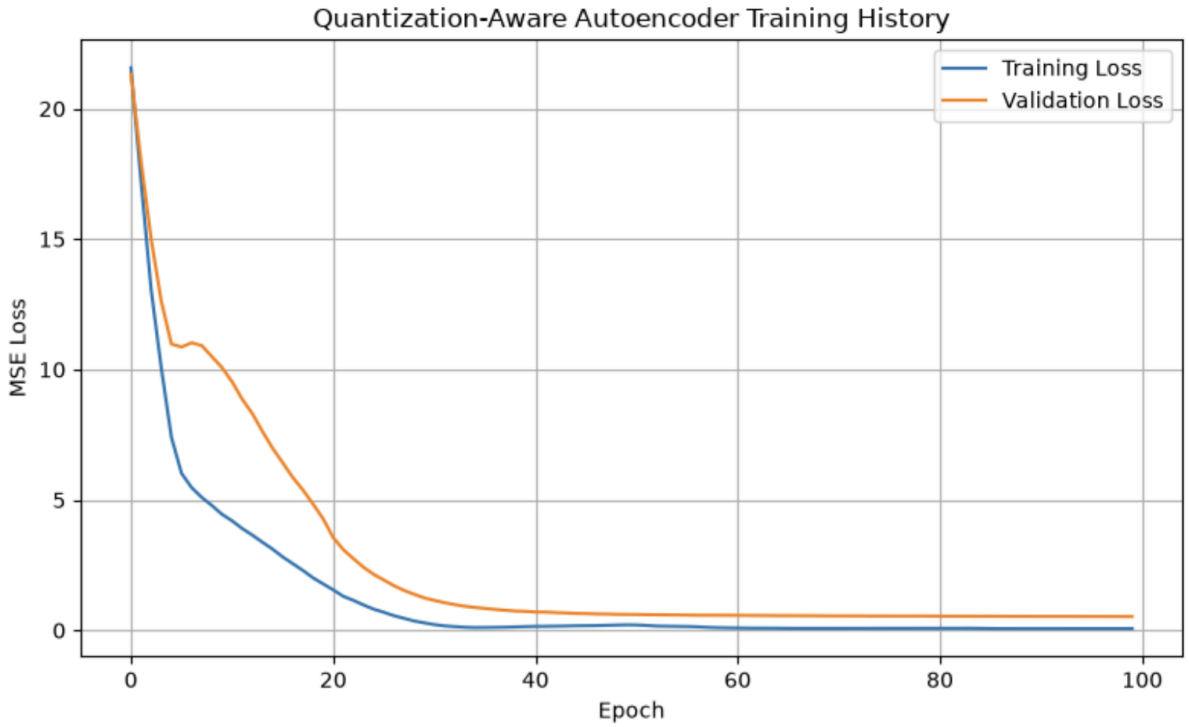
The classifier output should not always be trusted. It is more reliable when the board is mounted on a similar fan in the same position and orientation used during training. It may be unreliable when the board is placed on a table, moved, mounted differently, exposed to external vibration, or given inputs unlike the training data. Therefore, classification confidence should be considered together with the anomaly score and results from multiple consecutive windows.

Section 2

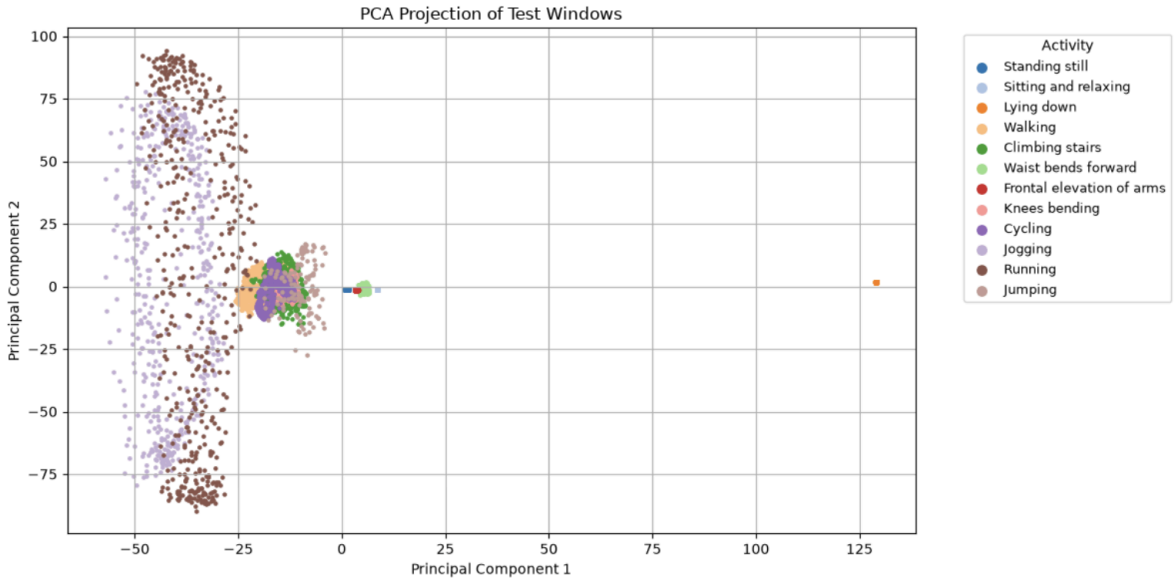
clear and labeled screenshots of the following:

• Training and validation loss curves

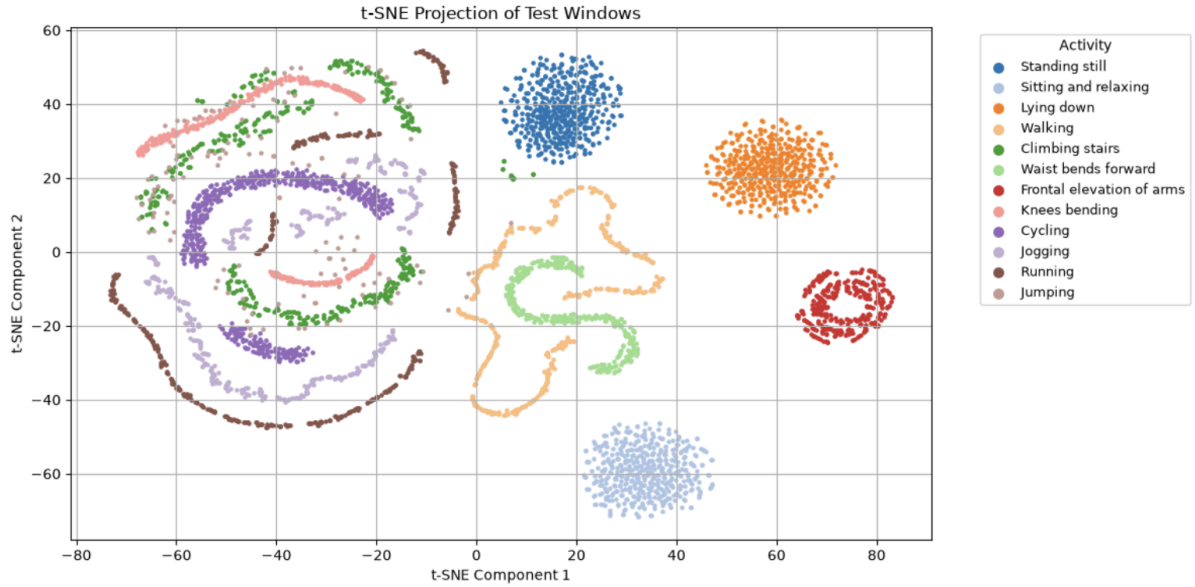




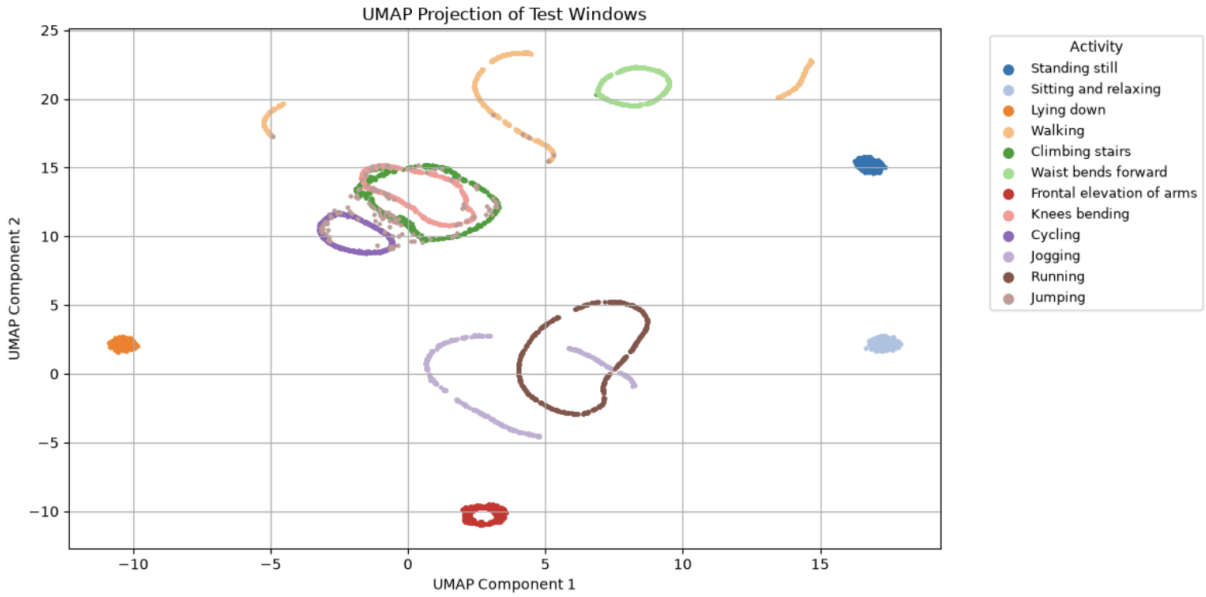
• PCA visualization



• t-SNE visualization

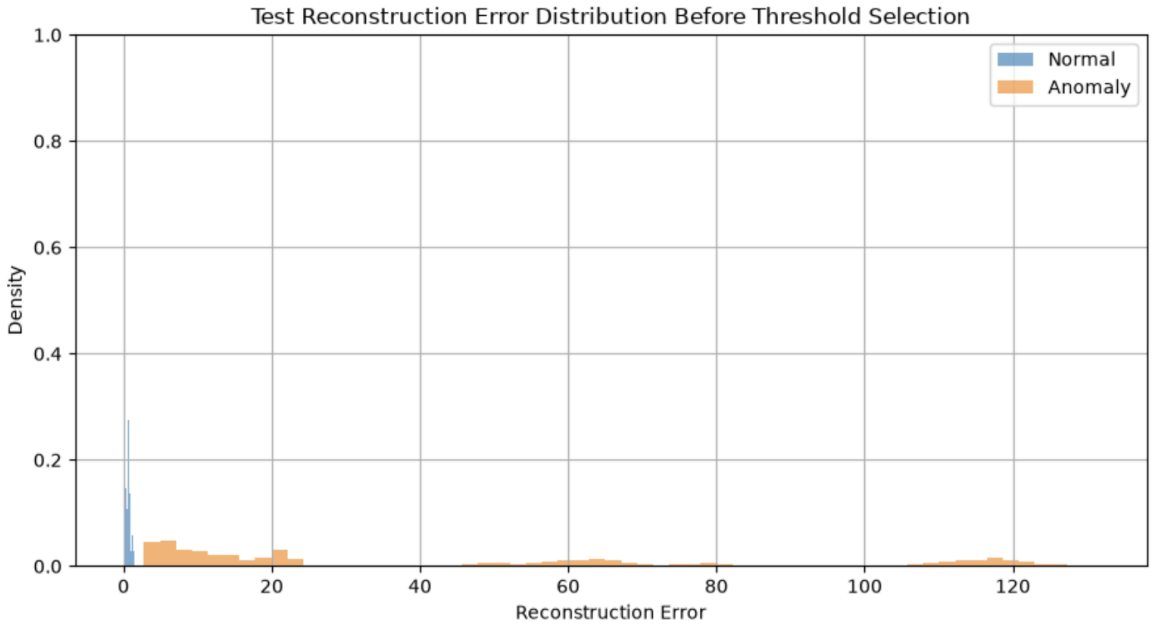


• UMAP visualization



• Reconstruction error distribution

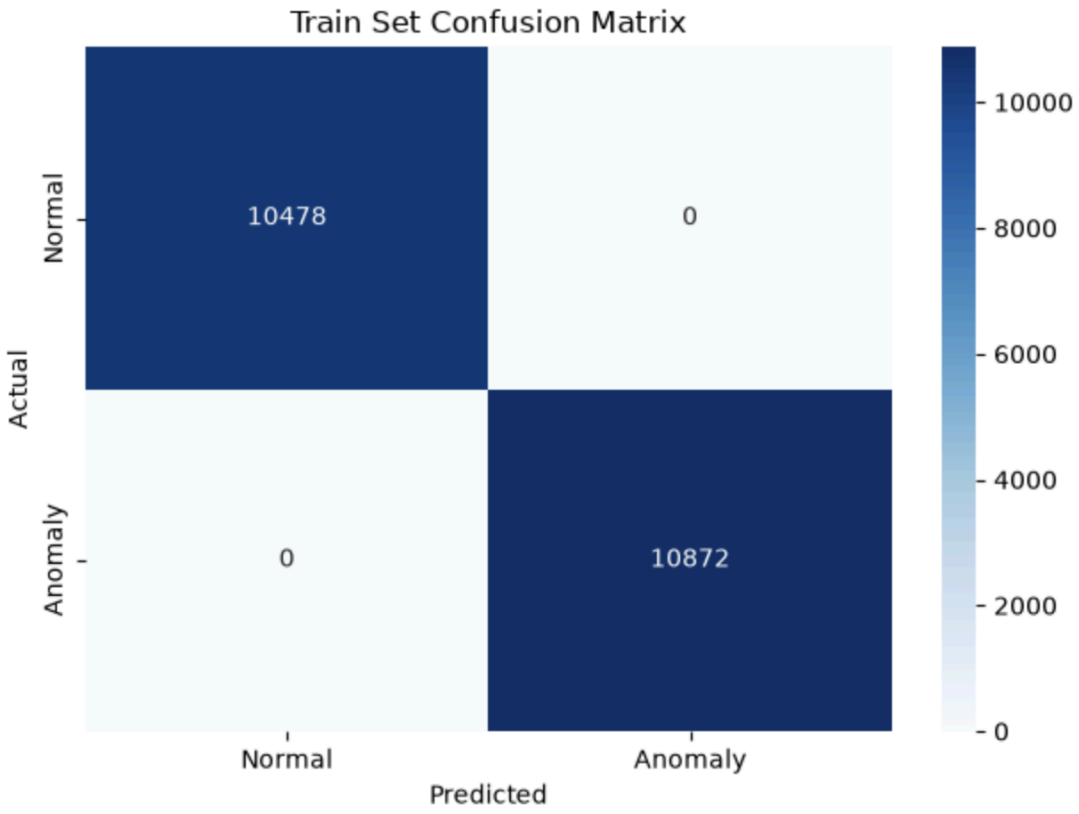
```
Train normal       mean=0.115544, stderr=0.001652, min=0.008864, max=1.125529
Train anomaly      mean=42.957478, stderr=0.375519, min=2.709212, max=130.675186
Test normal        mean=0.160632, stderr=0.004031, min=0.009338, max=1.484679
Test anomaly       mean=43.276653, stderr=0.627108, min=2.767601, max=131.525070
```

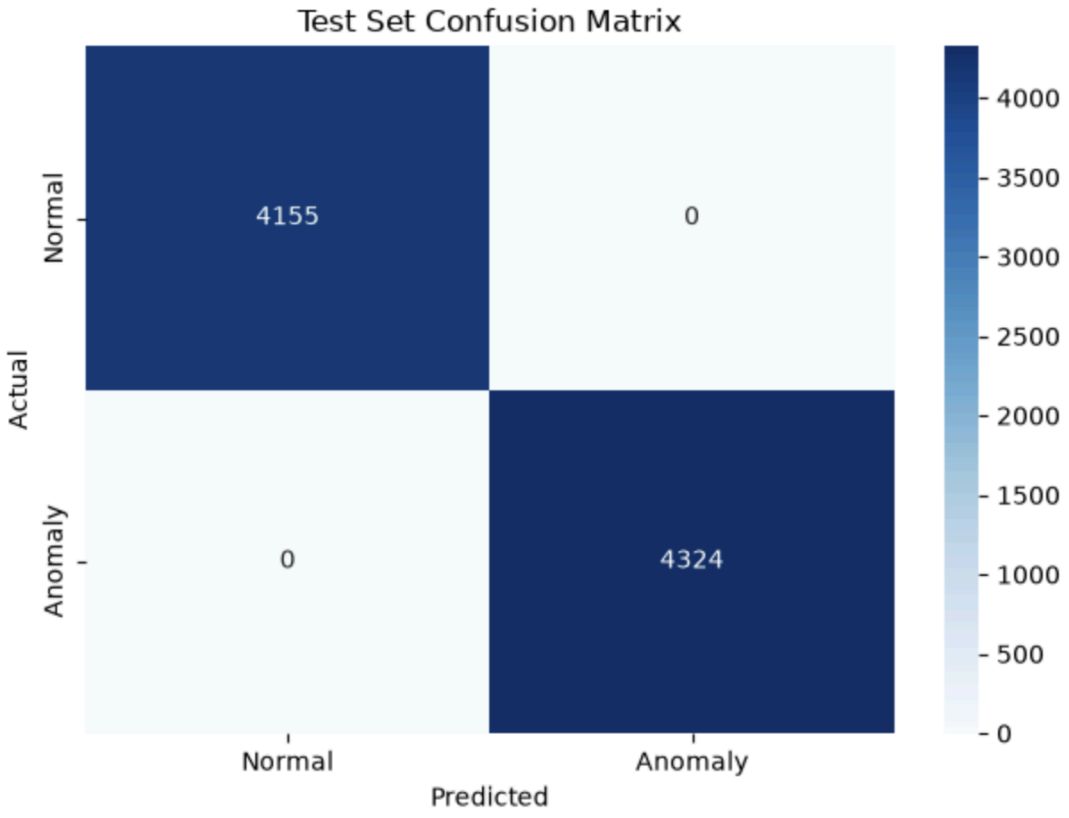


- Selected anomaly threshold

```
# Choose a reconstruction-error threshold after inspecting the distributions.
threshold = 2.0
```

- Confusion matrix

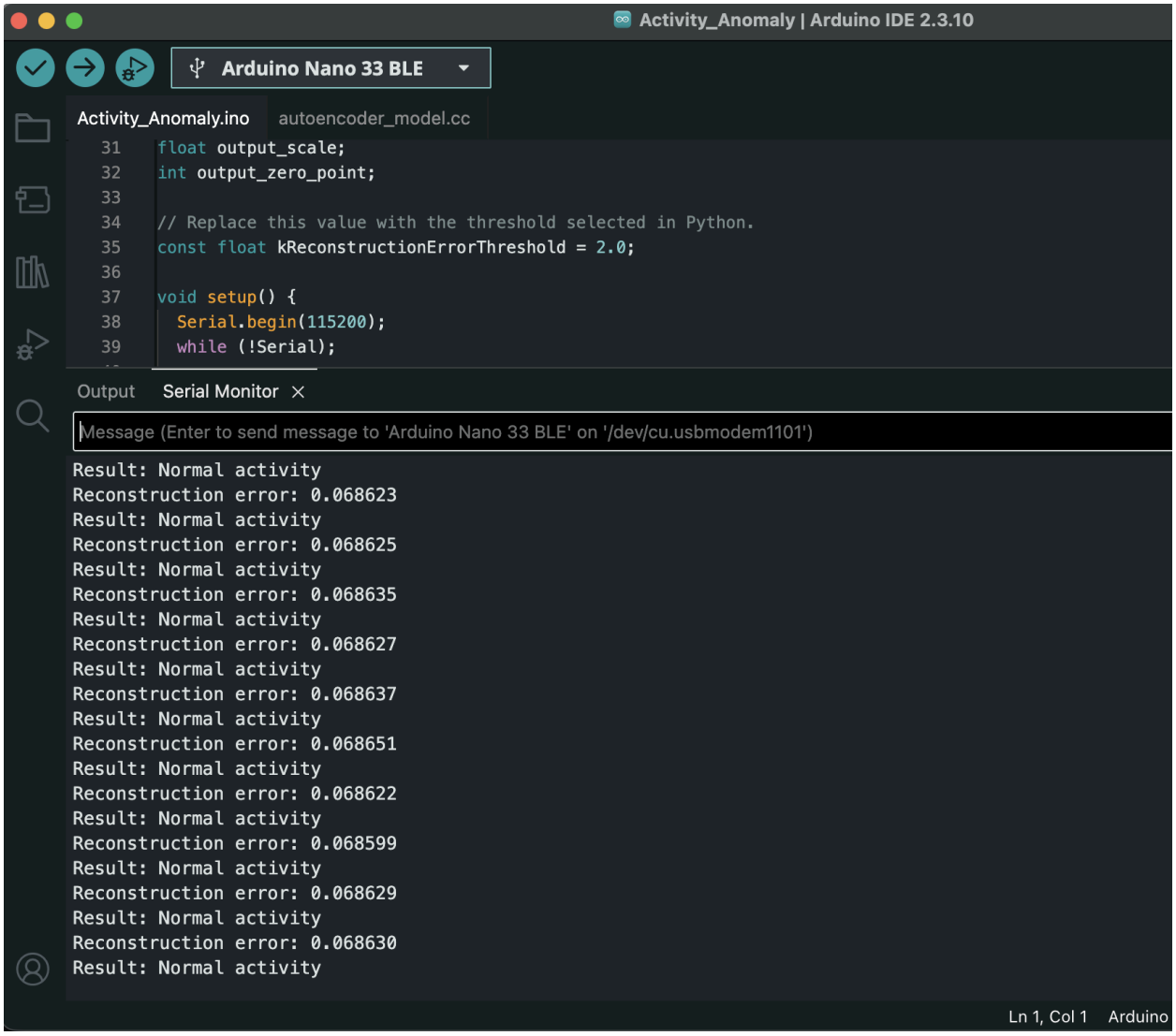

Train Set Confusion Matrix
Actual
Normal
Anomaly
10478
0
0
10872
Normal
Anomaly
Predicted
10000
8000
6000
4000
2000
0



• Classification report or evaluation metrics

```
Train set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00     10478
 Anomaly (1)       1.00      1.00      1.00     10872

    accuracy                           1.00     21350
   macro avg       1.00      1.00      1.00     21350
weighted avg       1.00      1.00      1.00     21350

Test set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00      4155
 Anomaly (1)       1.00      1.00      1.00      4324

    accuracy                           1.00      8479
   macro avg       1.00      1.00      1.00      8479
weighted avg       1.00      1.00      1.00      8479
```

• Arduino serial monitor output showing reconstruction error and normal/anomaly detection results



Section 3

Answer the following briefly:

• What threshold did you choose for anomaly detection, and why?

I chose **2.0** as the anomaly-detection threshold based on the training reconstruction-error distributions. The maximum reconstruction error among the normal training windows was approximately **1.13**, while the minimum error among the anomalous training windows was approximately **2.71**. Therefore, 2.0 lies within the gap between the two distributions. Windows with reconstruction errors greater than 2.0 are classified as anomalies.

• How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder was trained only on normal activity windows, so it learned to reconstruct normal motion patterns accurately. Normal inputs therefore tend to produce small reconstruction errors. Anomalous activities differ from the learned normal patterns and are usually reconstructed less accurately, producing larger errors. The reconstruction error is compared with the selected threshold to classify each window as normal or anomalous.

• What information from the Python notebook must be preserved when deploying the model to Arduino?
The Arduino deployment must preserve the generated int8 model bytes, the input-window size and shape, the sensor channels and their ordering, the sampling assumptions, the int8 input and output quantization parameters, the reconstruction-error calculation, and the selected anomaly threshold of 2.0. In this lab, the model expects a flattened 300-value input representing 100 samples from three accelerometer axes.

• Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?
The model was trained on data prepared with a specific window size, sensor-channel order, data type, and quantization procedure. The Arduino sketch must reproduce these assumptions so that its input has the same meaning and numerical representation as the training input. If the sampling rate, axis order, window length, flattening order, scaling, or quantization differs, the model receives out-of-distribution inputs and may produce incorrect reconstruction errors and anomaly decisions.

• What are the main limitations of this anomaly detection method when deployed on a microcontroller?
The method has several limitations. Its accuracy depends heavily on whether the training data represents the full range of normal activities and deployment conditions. Unseen but legitimate normal behavior may be falsely classified as anomalous, while some anomalies similar to normal patterns may be missed. Performance is also sensitive to sensor placement, orientation, noise, sampling rate, and threshold selection. In addition, int8 quantization and the limited memory and computation available on a microcontroller can reduce model capacity and numerical precision. Finally, a high reconstruction error indicates that an input is unusual, but it does not identify the specific type or cause of the anomaly.